

MOVIE STREAMING DATABASE

1. Project Overview

The Movie Streaming Database is a relational database that stores information about users, movies, genres, subscriptions, watch history, and user ratings and reviews. It is built and tested in MySQL Workbench and supports a movie-streaming application whose front end is being developed on top of it.

SQL is responsible for storing and retrieving the structured data. The same data can be used to look up movie information, analyze user preferences, calculate popularity and rating-based results, and later generate personalized recommendations.

The design consists of 7 tables, documented with an ER diagram, a schema diagram, an entity-relationship description, and a functional dependency and normalization proof. The database is loaded with sample data so that every table can be queried and demonstrated.

2. Objectives

- Store user and movie information.
- Organize movies by genres.
- Manage user subscription plans and their validity periods.
- Record what each user watches, when, and for how long.
- Store user ratings and reviews.
- Analyze user preferences and content popularity.
- Support movie recommendations built on this data.
- Demonstrate a practical, normalized relational database application.

3. Main Database Tables

Table	Purpose	Key columns	Sample rows
Users	Stores registered user information.	user_id (PK), username, email, password, signup_date	50
Movies	Stores movie details such as title, release year, duration, language and catalogue rating.	movie_id (PK), title, release_year, duration, language, rating	50
Genres	Stores movie categories such as Action, Comedy and Drama.	genre_id (PK), genre_name	15
Movie_Genre	Connects movies and genres; handles the many-to-many relationship.	movie_id (FK), genre_id (FK) - composite PK	129

Table	Purpose	Key columns	Sample rows
Subscription	Stores each user's subscription plan and its start and end dates.	subscription_id (PK), user_id (FK), plan, start_date, end_date	45
Watch_History	Stores which movies each user watched, when, and how many minutes.	history_id (PK), user_id (FK), movie_id (FK), watch_date, watched_minutes, completed	68
Ratings	Stores the score and review a user gave a movie.	rating_id (PK), user_id (FK), movie_id (FK), score, review	56

4. Relationships

Relationship	Type	How it is implemented
Users - Subscription	1 : N	Subscription.user_id references Users.user_id. A user may have no subscription or several over time (renewals, plan changes); each subscription belongs to exactly one user.
Users - Movies (watches)	M : N	Resolved by Watch_History. A user can watch many movies and a movie can be watched by many users; the table also stores watch_date, watched_minutes and completed.
Users - Movies (rates)	M : N	Resolved by Ratings. A user can rate many movies and a movie can receive many ratings; each user rates a movie at most once (UNIQUE on user_id, movie_id).
Movies - Genres	M : N	Resolved by Movie_Genre. A movie belongs to one or more genres and a genre contains many movies; the pair (movie_id, genre_id) is the composite primary key.

Users and Movies share two separate many-to-many relationships (watches and rates). They are resolved into two bridge tables because each carries different attributes: viewing statistics in one, and score and review text in the other.

5. Sample Data and Integrity Rules

- All data is sample data created for demonstration. Emails use the example.com domain and the password column holds dummy strings, not real credentials.
- Movie titles are real; durations, ratings, reviews, subscriptions and viewing activity are illustrative.
- Every movie belongs to at least one genre.
- Every watch record and rating refers to an existing user and an existing movie (foreign keys).
- Watch dates fall after the user's signup date and inside the user's subscription period; each rating is for a movie that user finished watching.
- Five sample users have no subscription, which demonstrates that a subscription is optional for a user.

- CHECK constraints enforce valid values: release year of 1900 or later, positive duration, ratings and scores between 0 and 10, plan of Basic, Standard or Premium, and an end date after the start date.

6. Normalization

Every relation in the database (Users, Movies, Genres, Movie_Genre, Subscription, Watch_History and Ratings) satisfies First, Second and Third Normal Form. Because every determinant in each relation is a candidate key, the design also satisfies Boyce-Codd Normal Form (BCNF). The full step-by-step proof with functional dependencies is provided in the separate Functional Dependencies and Normalization document.

7. Running the Database

1. Open MySQL Workbench (MySQL 8.0.16 or newer, so that CHECK constraints are enforced) and connect to a local server.
2. Open **moviestreaming_database.sql** and run the whole script. It recreates the moviestreaming database, creates all 7 tables and loads the sample data.

The script begins with DROP DATABASE IF EXISTS moviestreaming, so it deletes any existing database with that name. Tables are created and loaded in dependency order: Users, Movies and Genres first, then Movie_Genre, Subscription, Watch_History and Ratings.

8. Why This Database Design Is Useful

Different information is stored in separate related tables instead of one large table. This reduces duplication, makes updates easier, maintains data integrity through keys and constraints, and allows efficient analysis using SQL joins and aggregate queries.

9. Future Scope

Subscriptions, watch history and watch duration are already part of the current design. The project can be expanded further into a complete intelligent recommendation platform:

1. Add favorites, a watchlist, likes and dislikes.
2. Add actor, director, country, production house and keyword tables.
3. Add content-based filtering using movie metadata.
4. Add collaborative filtering using user-item interaction data.
5. Add a machine-learning recommendation model and store the generated recommendations.
6. Add a recommendation history table to measure whether users interact with suggested movies.
7. Add stored procedures, triggers and scheduled jobs for automation, for example keeping Movies.rating in step with user ratings.
8. Add indexes and query optimization for large datasets.
9. Connect the database to the web front end through a backend API.

10. Add authentication with properly hashed passwords, role-based access, backups and database security.

10. Possible Future Tables

Table	Purpose
watchlist	Stores movies saved for later.
favorites	Stores favorite movies.
movie_cast	Connects actors with movies.
directors	Stores director information.
recommendations	Stores generated recommendations and scores.